Etudiant : Adama Fall
Niveau : Ing1 en Systèmes d'Information et Données
Cours : Big Data | Enseignant : Dr Diadié Sow

Rapport : Hadoop distribué et MapReduce

Ce rapport présente la construction d'un cluster Hadoop 3.4.1 sur Docker Compose, composé d'un NameNode, de trois DataNodes, d'un ResourceManager et de trois NodeManagers, avec une réplication HDFS fixée à 2. Le jeu de données, `transactions.csv` (100 000 transactions synthétiques), est stocké dans HDFS, puis exploité par quatre traitements MapReduce écrits en Python avec Hadoop Streaming. Enfin, le comportement du cluster est analysé lors de la panne d'un DataNode et d'un NodeManager.

Partie A — Construire et vérifier le cluster

A.1 Architecture du cluster

Le tableau ci-dessous présente l'architecture du cluster. Le NameNode gère les métadonnées de HDFS ,il sait quels blocs composent chaque fichier et où ils se trouvent, mais ne stocke aucun bloc. Les trois DataNodes stockent les blocs de données et signalent régulièrement leur état au NameNode. Le ResourceManager répartit le travail : il choisit les NodeManagers qui ont les ressources nécessaires pour exécuter les tâches map et reduce. Les trois NodeManagers exécutent ensuite ces tâches dans des containers YARN. Chaque service tourne dans son propre conteneur Docker, ce qui rend les DataNodes et les NodeManagers identifiables séparément.

Service	Conteneur	Role	Port exposé
NameNode	hadoop-namenode	Métadonnées HDFS	9870
DataNode 1 à 3	hadoop-datanode1 à 3	Stockage des blocs	aucun
ResourceManager	hadoop-resourcemanager	Allocation des ressources	8088
NodeManager 1 à 3	hadoop-nodemanager1 à 3	Exécution des containers YARN	aucun

A.2 Fichiers produits et justifications

La construction du cluster Hadoop repose sur plusieurs fichiers de configuration. Nous en avons créé six : un Dockerfile, qui construit l'image commune à tous les nœuds, quatre fichiers XML (core-site.xml, hdfs-site.xml, mapred-site.xml et yarn-site.xml), qui configurent respectivement HDFS, MapReduce et YARN, et un fichier compose.yaml, qui déclare et relie les huit conteneurs du cluster. Chaque fichier est présenté ci-dessous, avec le choix technique qui le justifie.

a) core-site.xml

conf > core-site.xml

```xml
<configuration>
    <property>
        <name>fs.defaultFS</name>
        <value>hdfs://namenode:9000</value>
    </property>
</configuration>
```

Ce fichier définit fs.defaultFS, qui désigne le NameNode (hdfs://namenode:9000) comme point d'entrée du système de fichiers. Tous les services du cluster savent ainsi où contacter HDFS.

b) hdfs-site.xml

```xml
<configuration>

    <!-- ============================================================ -->
    <!-- Réplication et taille de bloc                                -->
    <!-- ============================================================ -->

    <!-- Chaque bloc est copié sur 2 DataNodes -->
    <property>
        <name>dfs.replication</name>
        <value>2</value>
    </property>

    <!-- Bloc de 1 Mio = 1 048 576 octets  -->
    <property>
        <name>dfs.blocksize</name>
        <value>1048576</value>
    </property>

    <!-- ============================================================ -->
    <!-- Stockage (monté sur des volumes Docker nommés dans le compose) -->
    <!-- ============================================================ -->

    <!-- Métadonnées du NameNode -->
    <property>
        <name>dfs.namenode.name.dir</name>
        <value>file:///tmp/hadoop-root/dfs/name</value>
    </property>
```

```xml
    <!-- Blocs de données des DataNodes -->
    <property>
        <name>dfs.datanode.data.dir</name>
        <value>file:///tmp/hadoop-root/dfs/data</value>
    </property>

    <!-- ============================================================ -->
    <!-- Détection de panne (Partie D)                                -->
    <!-- Délai = 2 x 5 s + 10 x 3 s = 40 s    -->
    <!-- ============================================================ -->

    <!-- Intervalle entre deux heartbeats d'un DataNode (secondes) -->
    <property>
        <name>dfs.heartbeat.interval</name>
        <value>3</value>
    </property>

    <!-- Intervalle de vérification des heartbeats par le NameNode (ms) -->
    <property>
        <name>dfs.namenode.heartbeat.recheck-interval</name>
        <value>5000</value>
    </property>

</configuration>
```

La réplication est fixée à 2 comme demandé par le sujet, et la taille de bloc à 1 Mio (1 048 576 octets) afin que transactions.csv (6,78 Mio) . Les répertoires de stockage du NameNode et des DataNodes seront montés sur des volumes Docker nommés pour assurer la persistance. Pour la Partie D, la

détection de panne est ramenée à 40 s (2 × 5 s + 10 × 3 s) au lieu de 10 min 30 s par défaut, afin de faciliter les observations.

c) mapred-site.xml

```xml
<configuration>


    <!-- Les jobs MapReduce s'exécutent sur YARN  -->
    <property>
        <name>mapreduce.framework.name</name>
        <value>yarn</value>
    </property>

    <!-- ============================================================= -->
    <!-- Classpath                                                     -->
    <!-- ============================================================= -->

    <!-- Emplacement des classes MapReduce que les containers YARN
         doivent charger (AppMaster, mappers, reducers) -->
    <property>
        <name>mapreduce.application.classpath</name>
        <value>$HADOOP_MAPRED_HOME/share/hadoop/mapreduce/*:$HADOOP_MAPRED_HOME/share/hadoop/mapreduce/lib/*</value>
    </property>

</configuration>
```

Les jobs MapReduce s'exécutent sur YARN (mapreduce.framework.name = yarn) . Le classpath indique aux containers YARN où trouver les classes MapReduce nécessaires à l'AppMaster, aux mappers et aux reducers.

d) yarn-site.xml

conf > yarn-site.xml

```xml
<configuration>
    <!-- ResourceManager                                               -->
    <!-- ============================================================= -->

    <!-- Nom d'hôte du ResourceManager (nom du service dans le compose) -->
    <property>
        <name>yarn.resourcemanager.hostname</name>
        <value>resourcemanager</value>
    </property>

    <!-- ============================================================= -->
    <!-- NodeManager                                                   -->
    <!-- ============================================================= -->

    <!-- Service de shuffle : transfère les sorties des mappers
         vers les reducers (indispensable à MapReduce) -->
    <property>
        <name>yarn.nodemanager.aux-services</name>
        <value>mapreduce_shuffle</value>
    </property>

    <!-- Variables d'environnement que les NodeManagers transmettent
         aux containers YARN -->
    <property>
        <name>yarn.nodemanager.env-whitelist</name>
        <value>JAVA_HOME,HADOOP_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_CONF_DIR,HADOOP_YARN_HOME,HADOOP_MAPRED_HOME,PATH</value>
    </property>

</configuration>
```

Ce fichier indique l'adresse du ResourceManager, active le service de shuffle (mapreduce_shuffle), indispensable pour transférer les sorties des mappers vers les reducers, et définit les variables d'environnement que les NodeManagers transmettent aux containers YARN.

e) Dockerfile :

```dockerfile
FROM apache/hadoop:3.4.1

USER root

# Utiliser les archives CentOS 7
RUN sed -i 's|^mirrorlist=|#mirrorlist=|g' /etc/yum.repos.d/CentOS-Base.repo && \
    sed -i 's|^#baseurl=http://mirror.centos.org/centos/$releasever/|baseurl=https://vault.centos.org/7.9.2009/|g' /etc/yu

# Installer Python 3
RUN yum --disablerepo=epel -y install python3 && \
    yum clean all

# Configuration embarquée : même config sur tous les nœuds
COPY conf/*.xml /opt/hadoop/etc/hadoop/
```

Nous utilisons l'image officielle apache/hadoop:3.4.1, qui fournit Hadoop et Java déjà configurés, avec la version imposée par le sujet. Les services s'exécutent en root pour éviter les problèmes de permissions sur les volumes Docker. Comme CentOS 7 est en fin de vie, ses dépôts sont redirigés vers vault.centos.org afin de pouvoir installer Python 3, nécessaire à Hadoop Streaming sur tous les nœuds. Les fichiers de configuration sont copiés dans l'image pour que chaque nœud ait la même configuration, ce qui rend le cluster reproductible .

f) compose.yaml

```yaml
services:

  # ---------------- HDFS ----------------

  namenode:
    image: hadoop-python:3.4.1
    container_name: hadoop-namenode
    hostname: namenode
    user: root
    command:
      - bash
      - -c
      - "[ -d /tmp/hadoop-root/dfs/name/current ] || hdfs namenode -format -force -nonInteractive; hdfs namenode"
    ports:
      - "9870:9870"
    volumes:
      - namenode_data:/tmp/hadoop-root/dfs/name
      - ./data:/data
      - ./mapreduce:/mapreduce

  datanode1:
    image: hadoop-python:3.4.1
    container_name: hadoop-datanode1
    hostname: datanode1
    user: root
```

```
    command: ["hdfs", "datanode"]
    depends_on:
      - namenode
    volumes:
      - datanode1_data:/tmp/hadoop-root/dfs/data

  ▷Run Service
  datanode2:
    image: hadoop-python:3.4.1
    container_name: hadoop-datanode2
    hostname: datanode2
    user: root
    command: ["hdfs", "datanode"]
    depends_on:
      - namenode
    volumes:
      - datanode2_data:/tmp/hadoop-root/dfs/data

  ▷Run Service
  datanode3:
    image: hadoop-python:3.4.1
    container_name: hadoop-datanode3
    hostname: datanode3
    user: root
    command: ["hdfs", "datanode"]
```

```
    depends_on:
      - namenode
    volumes:
      - datanode3_data:/tmp/hadoop-root/dfs/data

  # ---------------- YARN ----------------

  ▷Run Service
  resourcemanager:
    image: hadoop-python:3.4.1
    container_name: hadoop-resourcemanager
    hostname: resourcemanager
    user: root
    command: ["yarn", "resourcemanager"]
    ports:
      - "8088:8088"
    depends_on:
      - namenode

  ▷Run Service
  nodemanager1:
    image: hadoop-python:3.4.1
    container_name: hadoop-nodemanager1
    hostname: nodemanager1
    user: root
    command: ["yarn", "nodemanager"]
```

```
    depends_on:
      - resourcemanager

  ▷Run Service
  nodemanager2:
    image: hadoop-python:3.4.1
    container_name: hadoop-nodemanager2
    hostname: nodemanager2
    user: root
    command: ["yarn", "nodemanager"]
    depends_on:
      - resourcemanager

  ▷Run Service
  nodemanager3:
    image: hadoop-python:3.4.1
    container_name: hadoop-nodemanager3
    hostname: nodemanager3
    user: root
    command: ["yarn", "nodemanager"]
    depends_on:
      - resourcemanager
```

```
volumes:
  namenode_data:
  datanode1_data:
  datanode2_data:
  datanode3_data:
```

Le compose.yaml déclare 1 NameNode, 3 DataNodes, 1 ResourceManager et 3 NodeManagers, tous basés sur la même image hadoop-python:3.4.1, construite au préalable avec docker build à partir du Dockerfile. Chaque nœud possède son propre hostname et son propre nom de conteneur, ce qui rend les trois DataNodes et les trois NodeManagers identifiables séparément. Chaque DataNode et le NameNode disposent de leur propre volume , ce qui rend le stockage persistant : les données survivent à un docker compose down. Les interfaces web sont exposées sur les ports 9870 (NameNode) et 8088 (ResourceManager). Le NameNode formate son espace de noms uniquement au premier démarrage, puis monte le jeu de données sur /data ainsi que le dossier mapreduce contenant les mappers et reducers.

A.3 Procédure de démarrage

Pour démarrer le cluster, il faut d'abord construire l'image à partir du Dockerfile, puis lancer les conteneurs avec Docker Compose, et enfin vérifier que tous les conteneurs ont bien démarré en consultant leur statut.

1. Construction de l'image : cette commande crée l'image hadoop-python:3.4.1, commune à tous les nœuds.

   docker build -t hadoop-python:3.4.1 .

2. Lancement des conteneurs : cette commande démarre les huit services en arrière-plan.

   docker compose up -d

3. Vérification du statut : les huit conteneurs doivent apparaître avec le statut Up.

   docker compose ps

A.4 Vérification du cluster

1. Statut des conteneurs
   En faisant un docker compose ps on voit que les huit conteneurs sont à l'état Up, et les ports 9870 et 8088 sont exposés.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker compose ps
NAME                    IMAGE                  COMMAND                  SERVICE           CREATED          STATUS          PORTS
hadoop-datanode1        hadoop-python:3.4.1    "/usr/local/bin/dumb…"   datanode1         30 minutes ago   Up 30 minutes
hadoop-datanode2        hadoop-python:3.4.1    "/usr/local/bin/dumb…"   datanode2         30 minutes ago   Up 30 minutes
hadoop-datanode3        hadoop-python:3.4.1    "/usr/local/bin/dumb…"   datanode3         30 minutes ago   Up 30 minutes
hadoop-namenode         hadoop-python:3.4.1    "/usr/local/bin/dumb…"   namenode          30 minutes ago   Up 30 minutes   0.0.0.0:
9870->9870/tcp
hadoop-nodemanager1     hadoop-python:3.4.1    "/usr/local/bin/dumb…"   nodemanager1      30 minutes ago   Up 30 minutes
hadoop-nodemanager2     hadoop-python:3.4.1    "/usr/local/bin/dumb…"   nodemanager2      30 minutes ago   Up 30 minutes
hadoop-nodemanager3     hadoop-python:3.4.1    "/usr/local/bin/dumb…"   nodemanager3      30 minutes ago   Up 30 minutes
hadoop-resourcemanager  hadoop-python:3.4.1    "/usr/local/bin/dumb…"   resourcemanager   30 minutes ago   Up 30 minutes   0.0.0.0:
8088->8088/tcp
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

2. Rapport HDFS

On exécute `hdfs dfsadmin -report` depuis le NameNode pour consulter l'état global du cluster HDFS :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfsadmin -report
Configured Capacity: 3243303530496 (2.95 TB)
Present Capacity: 3018441940992 (2.75 TB)
DFS Remaining: 3018422857728 (2.75 TB)
DFS Used: 19083264 (18.20 MB)
DFS Used%: 0.00%
Replicated Blocks:
        Under replicated blocks: 0
        Blocks with corrupt replicas: 0
        Missing blocks: 0
        Missing blocks (with replication factor 1): 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0
Erasure Coded Block Groups:
        Low redundancy block groups: 0
        Block groups with corrupt internal blocks: 0
        Missing block groups: 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0

-------------------------------------------------
Live datanodes (3):
```

Le rapport indique `Live datanodes (3)` : les trois DataNodes sont enregistrés auprès du NameNode et actifs. Les compteurs de blocs sous-répliqués, corrompus ou manquants sont tous à 0, ce qui montre que HDFS est sain. Le cluster affiche une capacité configurée de 2,95 To.

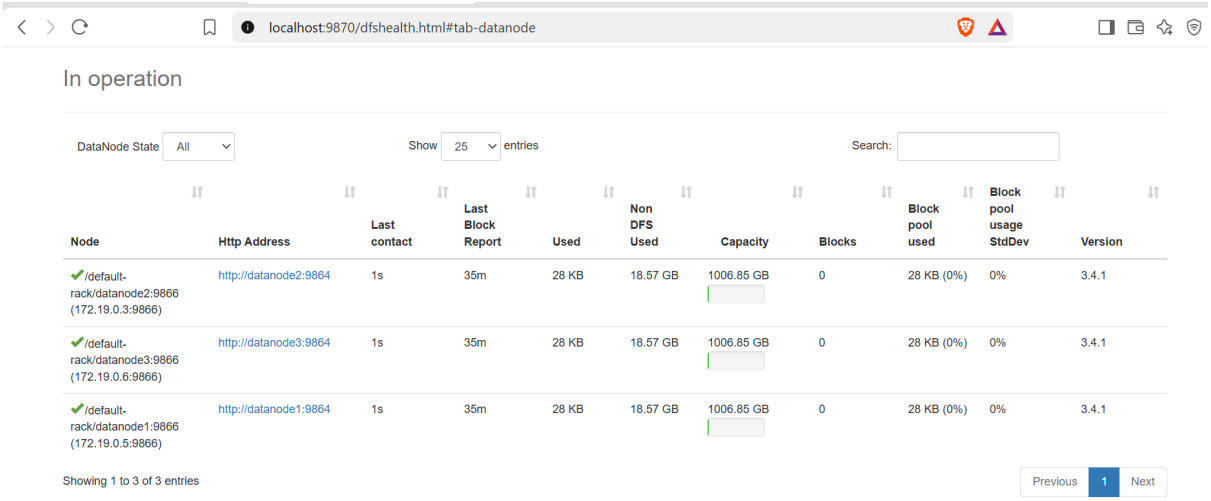
3. Nœuds YARN

En lançant la commande yarn node - -list ,on voit les trois nodemanager avec comme statut running :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-resourcemanager yarn
 node -list
2026-11-06 00:51:28 INFO  DefaultNoHARMFailoverProxyProvider:64 - Connecting to ResourceManager at resou
rcemanager/172.19.0.4:8032
Total Nodes:3
         Node-Id             Node-State Node-Http-Address       Number-of-Running-Containers
nodemanager2:46419               RUNNING nodemanager2:8042                                  0
nodemanager3:35347               RUNNING nodemanager3:8042                                  0
nodemanager1:37549               RUNNING nodemanager1:8042                                  0
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

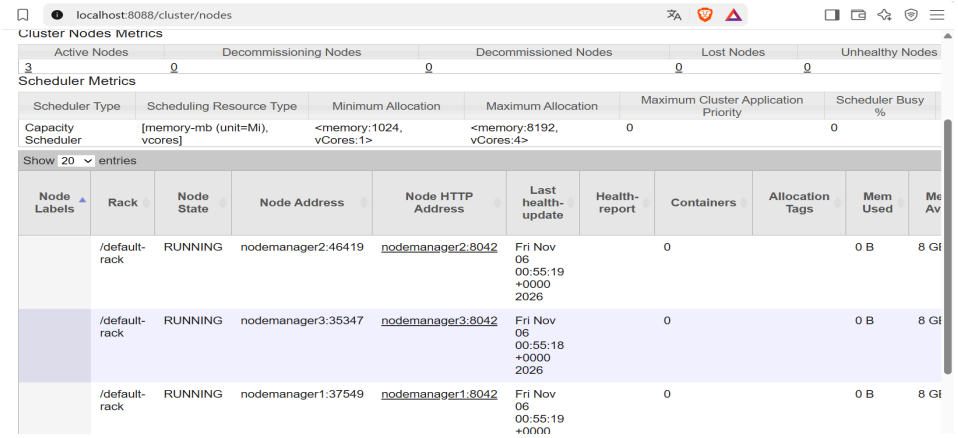
4. Interface NameNode (port 9870)

L'onglet Datanodes montre datanode1, datanode2 et datanode3 en service, chacun avec une capacité de 1006,85 Go et un dernier contact de 1 s.



5. Interface ResourceManager (port 8088)

La page Nodes indique 3 nœuds actifs (Active Nodes = 3) et aucun nœud perdu ou défaillant : nodemanager1, nodemanager2 et nodemanager3 sont RUNNING.



Le cluster est opérationnel : 3 DataNodes actifs côté HDFS et 3 NodeManagers enregistrés côté YARN.

A.5 Explications demandées

- Rôle de chaque service

Le NameNode gère les métadonnées de HDFS (fichiers, blocs, emplacements) sans stocker de blocs. Les DataNodes stockent les blocs et envoient des heartbeats au NameNode. Le ResourceManager répartit les ressources de calcul entre les jobs. Les NodeManagers exécutent les tâches dans des containers YARN sur leur nœud.

- Conteneur Docker et container YARN

Un conteneur Docker est un environnement isolé qui simule une machine du cluster (par exemple hadoop-nodemanager1). Un container YARN est une réservation de ressources sur un NodeManager, dans laquelle s'exécute une tâche comme un mapper ou un reducer. Les containers YARN tournent donc à l'intérieur des conteneurs Docker des NodeManagers.

- ResourceManager, NodeManager, ApplicationMaster

Le ResourceManager reçoit les jobs et choisit les NodeManagers qui exécuteront les tâches. Le NodeManager démarre les containers demandés et surveille leur consommation. L'ApplicationMaster, lancé une fois par job, demande des containers au ResourceManager, fait démarrer les mappers et reducers, puis suit leur avancement.

Partie B — Importer les données dans HDFS

- Création du répertoire /assignment/transactions

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfs -mkdir -p /assignment/transactions
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

- Vérification du fichier source

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode ls  /data
transactions.csv
```

Le fichier transactions.csv est visible dans le conteneur NameNode (monté sur /data)

- Importation du fichier dans HDFS

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfs -put /data/transactions.csv /
assignment/transactions/
```

Le fichier est importé avec hdfs dfs -put. La commande est lancée depuis le NameNode, mais les blocs sont écrits sur les DataNodes.

- Vérification de l'import

```
PS C:\Users\pc\Desktodocker exec hadoop-namenode hdfs dfs -ls /assignment/transactions
Found 1 items
-rw-r--r--   2 root supergroup    7109550 2026-11-06 21:40 /assignment/transactions/transactions.csv
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

hdfs dfs -ls montre le fichier de 7 109 550 octets (6,78 Mio) avec un facteur de réplication de 2.

- Blocs, répliques et emplacements

hdfs fsck indique 7 blocs, chacun avec Live_repl=2. Six blocs font 1 048 576 octets (1 Mio) et le dernier 818 094 octets, ce qui totalise 7 109 550 octets. Chaque bloc est réparti sur deux DataNodes différents (adresses 172.19.0.3, 172.19.0.5 et 172.19.0.6), soit 14 répliques au total. Le statut du fichier est OK. Comme le montre les captures suivantes :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs fsck /assignment/transactions/tra
nsactions.csv -files -blocks -locations
Connecting to namenode via http://namenode:9870/fsck?ugi=root&files=1&blocks=1&locations=1&path=%2Fassignment%2Ftransactions%2Ftran
sactions.csv
FSCK started by root (auth:SIMPLE) from /172.19.0.2 for path /assignment/transactions/transactions.csv at Fri Nov 06 21:44:50 UTC 2
026
```

```
/assignment/transactions/transactions.csv 7109550 bytes, replicated: replication=2, 7 block(s):  OK
0. BP-165148381-172.19.0.2-1793924235033:blk_1073741832_1008 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.6:9866,DS-b
d787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.5:9866,DS-7a1c624b-ddcd-49c3-9933-536a6b58dd0c,DISK]]
1. BP-165148381-172.19.0.2-1793924235033:blk_1073741833_1009 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.3:9866,DS-4
3101419-2bd0-43bf-a86d-d52b28cacf2c,DISK], DatanodeInfoWithStorage[172.19.0.6:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK]]
2. BP-165148381-172.19.0.2-1793924235033:blk_1073741834_1010 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.6:9866,DS-b
d787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.5:9866,DS-7a1c624b-ddcd-49c3-9933-536a6b58dd0c,DISK]]
3. BP-165148381-172.19.0.2-1793924235033:blk_1073741835_1011 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.6:9866,DS-b
d787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.3:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
4. BP-165148381-172.19.0.2-1793924235033:blk_1073741836_1012 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.5:9866,DS-7
a1c624b-ddcd-49c3-9933-536a6b58dd0c,DISK], DatanodeInfoWithStorage[172.19.0.6:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK]]
5. BP-165148381-172.19.0.2-1793924235033:blk_1073741837_1013 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.6:9866,DS-b
d787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.3:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
6. BP-165148381-172.19.0.2-1793924235033:blk_1073741838_1014 len=818094 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.6:9866,DS-bd
787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.5:9866,DS-7a1c624b-ddcd-49c3-9933-536a6b58dd0c,DISK]]
```

- Pourquoi 100 000 lignes ne garantissent pas plusieurs blocs ?

Le nombre de blocs dépend de la taille du fichier en octets, pas du nombre de lignes. Avec la taille par défaut de 128 Mio, transactions.csv (6,78 Mio) tiendrait dans un seul bloc. En ramenant la taille à 1 Mio, il est découpé en 7 blocs.

Partie C — Quatre traitements MapReduce

Quatre traitements MapReduce sont réalisés avec Hadoop Streaming et Python 3 : chiffre d'affaires par pays, par catégorie, par moyen de paiement et par mois. Tous suivent la même méthode : mapper et reducer, test local, exécution sur YARN, récupération de la sortie HDFS, puis comparaison avec une validation indépendante. Les montants sont calculés en centimes entiers .

C.1 Principes communs aux quatre jobs

L'en-tête du CSV est ignoré dans le mapper (test transaction_id == "transaction_id"), tout comme les lignes qui n'ont pas 9 champs. Entre le map et le reduce, Hadoop trie les paires par clé (shuffle and sort), ce qui regroupe toutes les lignes d'une même clé : le reducer n'a donc qu'à cumuler tant que la clé ne change pas. En local, sort simule ce tri.

1. **Job 1 : chiffre d'affaires par pays**

Pour calculer le chiffre d'affaires par pays, les étapes suivantes sont réalisées.

Le mapper ignore l'en-tête et les lignes mal formées, convertit le prix en centimes entiers et émet pays<TAB>montant en centimes.

mapreduce > chiffre_affaires_par_pays_mapper.py > ...

```python
#!/usr/bin/env python3
import sys

for transaction in sys.stdin:

    # Supprimer le retour à la ligne puis découper la ligne sur les virgules
    fields = transaction.strip().split(",")

    # Ignorer les lignes qui n'ont pas exactement 9 valeurs
    if len(fields) != 9:
        continue

    # Donner un nom à chaque élément de la liste
    transaction_id, date, country, city, product, category, quantity, unit_price, payment_method = fields

    # Ignorer l'en-tête du fichier
    if transaction_id == "transaction_id":
        continue

    # Les champs sont des chaînes : convertir la quantité en entier pour pouvoir calculer
    quantity = int(quantity)

    # Convertir le prix unitaire en centimes entiers (round évite les erreurs de flottants)
    unit_price_cents = round(float(unit_price) * 100)

    # Émettre la paire clé<TAB>valeur : pays et montant de la vente en centimes
    print(f"{country}\t{quantity * unit_price_cents}")
```

Le reducer additionne les montants tant que le pays ne change pas, grâce au tri des clés effectué par Hadoop avant la réduction.

mapreduce > chiffre_affaires_par_pays_reducer.py > ...

```python
#!/usr/bin/env python3
import sys

current_country = None
current_amount = 0

for line in sys.stdin:

    # Séparer la clé et la valeur sur la tabulation
    country, amount = line.strip().split("\t")

    # Convertir le montant en entier (centimes) pour pouvoir additionner
    amount = int(amount)

    if current_country == country:
        # Même pays que la ligne précédente : cumuler
        current_amount = current_amount + amount
    else:
        # Nouveau pays : afficher le total du pays précédent
        if current_country is not None:
            print(f"{current_country}\t{current_amount}")
        # Démarrer le cumul du nouveau pays
        current_country = country
        current_amount = amount

# Afficher le total du dernier pays
if current_country is not None:
    print(f"{current_country}\t{current_amount}")
```

La commande cat | mapper | sort | reducer simule le traitement : sort joue le rôle du tri par clé de Hadoop.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode bash -c "cat /data/transactions.csv | python3 /
mapreduce/chiffre_affaires_par_pays_mapper.py | sort | python3 /mapreduce/chiffre_affaires_par_pays_reducer.py"
Canada  1123162050
France  1120959850
Germany 1138438050
Morocco 1127850500
Senegal 1123917300
```

Lancement sur YARN. Le job est soumis avec Hadoop Streaming. Le ResourceManager lance un ApplicationMaster, qui démarre les mappers (un par bloc) puis le reducer.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming
-3.4.1.jar -files /mapreduce/chiffre_affaires_par_pays_mapper.py,/mapreduce/chiffre_affaires_par_pays_reducer.py -mapper "python3 chiffre_affaires_par_pays
_mapper.py" -reducer "python3 chiffre_affaires_par_pays_reducer.py" -input /assignment/transactions/transactions.csv -output /assignment/output/chiffre_aff
aires_par_pays
```

```
2026-11-08 11:29:49 INFO  Job:1681 - The url to track the job: http://resourcemanager:8088/proxy/application_1794125597256_0002/
2026-11-08 11:29:49 INFO  Job:1726 - Running job: job_1794125597256_0002
2026-11-08 11:30:01 INFO  Job:1747 - Job job_1794125597256_0002 running in uber mode : false
2026-11-08 11:30:01 INFO  Job:1754 -  map 0% reduce 0%
2026-11-08 11:30:17 INFO  Job:1754 -  map 14% reduce 0%
2026-11-08 11:30:20 INFO  Job:1754 -  map 29% reduce 0%
2026-11-08 11:30:27 INFO  Job:1754 -  map 43% reduce 0%
2026-11-08 11:30:28 INFO  Job:1754 -  map 57% reduce 0%
2026-11-08 11:30:29 INFO  Job:1754 -  map 71% reduce 0%
2026-11-08 11:30:30 INFO  Job:1754 -  map 100% reduce 0%
2026-11-08 11:30:36 INFO  Job:1754 -  map 100% reduce 100%
2026-11-08 11:30:37 INFO  Job:1765 - Job job_1794125597256_0002 completed successfully
```

Vérification des fichiers. Le dossier de sortie contient _SUCCESS (job réussi) et part-00000 (résultat du reducer).

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>  docker exec hadoop-namenode hdfs dfs -ls /assignment/output/chiffre_affaires_par_pays/
Found 2 items
-rw-r--r--   2 root supergroup          0 2026-11-08 11:30 /assignment/output/chiffre_affaires_par_pays/_SUCCESS
-rw-r--r--   2 root supergroup         93 2026-11-08 11:30 /assignment/output/chiffre_affaires_par_pays/part-00000
```

Résultat. Les totaux par pays, en centimes, sont lus dans HDFS.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>  docker exec hadoop-namenode hdfs dfs -cat /assignment/output/chiffre_affaires_par_pays/pa
rt-00000
Canada  1123162050
France  1120959850
Germany 1138438050
Morocco 1127850500
Senegal 1123917300
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Un script Python indépendant relit transactions.csv avec le module csv, recalcule les totaux en centimes avec un dictionnaire sans MapReduce . Les résultats sont identiques comme le montre la capture ci-dessous :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode python3 /mapreduce/validate.py pays
Canada  1123162050
France  1120959850
Germany 1138438050
Morocco 1127850500
Senegal 1123917300
```

2. **Job 2 : chiffre d'affaires par catégorie**

Pour calculer le chiffre d'affaires par catégorie, les étapes suivantes sont réalisées.

Le mapper ignore l'en-tête et les lignes mal formées, convertit le prix en centimes entiers et émet catégorie<TAB>montant en centimes.

mapreduce > chiffre_affaires_par_categorie_mapper.py > ...

```python
#!/usr/bin/env python3
import sys

for transaction in sys.stdin:

    # Supprimer le retour à la ligne puis découper la ligne sur les virgules
    fields = transaction.strip().split(",")

    # Ignorer les lignes qui n'ont pas exactement 9 valeurs
    if len(fields) != 9:
        continue

    # Donner un nom à chaque élément de la liste
    transaction_id, date, country, city, product, category, quantity, unit_price, payment_method = fields

    # Ignorer l'en-tête du fichier
    if transaction_id == "transaction_id":
        continue

    # Les champs sont des chaînes : convertir la quantité en entier pour pouvoir calculer
    quantity = int(quantity)

    # Convertir le prix unitaire en centimes entiers (round évite les erreurs de flottants)
    unit_price_cents = round(float(unit_price) * 100)

    # Émettre la paire clé<TAB>valeur : categorie et montant de la vente en centimes
    print(f"{category}\t{quantity * unit_price_cents}")
```

Le reducer additionne les montants tant que la catégorie ne change pas, grâce au tri des clés effectué par Hadoop avant la réduction.

mapreduce > chiffre_affaires_par_categorie_reducer.py > ...

```python
#!/usr/bin/env python3
import sys

current_category = None
current_amount = 0

for line in sys.stdin:

    # Séparer la clé et la valeur sur la tabulation
    category, amount = line.strip().split("\t")

    # Convertir le montant en entier (centimes) pour pouvoir additionner
    amount = int(amount)

    if current_category == category:
        # Même catégorie que la ligne précédente : cumuler
        current_amount = current_amount + amount
    else:
        # Nouvelle catégorie : afficher le total de la catégorie précédente
        if current_category is not None:
            print(f"{current_category}\t{current_amount}")
        # Démarrer le cumul de la nouvelle catégorie
        current_category = category
        current_amount = amount

# Afficher le total de la dernière catégorie
if current_category is not None:
    print(f"{current_category}\t{current_amount}")
```

La commande cat | mapper | sort | reducer simule un test local. Elle donne quatre catégories : Accessories, Electronics, Furniture et Stationery.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode bash -c "cat /data/transactions.csv | python3 /mapreduce/chiffre_affaires_par_categorie_mapper.py | sort | python3 /mapreduce/chiffre_affaires_par_categorie_reducer.py"
Accessories     450822000
Electronics     4485705000
Furniture       681075500
Stationery      16725250
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Lancement sur YARN. Le job est soumis avec Hadoop Streaming. Le ResourceManager lance un ApplicationMaster, qui démarre les mappers puis le reducer.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hadoop jar /opt/hadoop
/share/hadoop/tools/lib/hadoop-streaming-3.4.1.jar -files /mapreduce/chiffre_affaires_par_categorie_mapper.py,/mapr
educe/chiffre_affaires_par_categorie_reducer.py -mapper "python3 chiffre_affaires_par_categorie_mapper.py" -reducer
 "python3 chiffre_affaires_par_categorie_reducer.py" -input /assignment/transactions/transactions.csv -output /assi
gnment/output/chiffre_affaires_par_categorie
```

```
2026-11-08 12:24:02 INFO  Job:1754 -  map 100% reduce 100%
2026-11-08 12:24:02 INFO  Job:1765 - Job job_1794125597256_0003 completed successfully
```

Vérification des fichiers. Le dossier de sortie contient _SUCCESS et part-00000.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfs -ls /assignme
nt/output/chiffre_affaires_par_categorie
>>
Found 2 items
-rw-r--r--   2 root supergroup          0 2026-11-08 12:24 /assignment/output/chiffre_affaires_par_categorie/_SUCCE
SS
-rw-r--r--   2 root supergroup         85 2026-11-08 12:24 /assignment/output/chiffre_affaires_par_categorie/part-0
0000
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Résultat. Les totaux par catégorie, en centimes, sont lus dans HDFS.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfs -cat /assignm
ent/output/chiffre_affaires_par_categorie/part-00000
Accessories     450822000
Electronics     4485705000
Furniture       681075500
Stationery      16725250
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Validation indépendante. Le script validate.py categorie recalcule les totaux depuis transactions.csv, sans MapReduce. Ses résultats sont identiques à ceux du job.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode python3 /mapreduce/val
idate.py categorie
Accessories     450822000
Electronics     4485705000
Furniture       681075500
Stationery      16725250
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

3. Job 3 : chiffre d'affaires par moyen de paiement

Pour calculer le chiffre d'affaires par moyen de paiement, les étapes suivantes sont réalisées.

Le mapper ignore l'en-tête et les lignes mal formées, convertit le prix en centimes entiers et émet moyen de paiement<TAB>montant en centimes.

mapreduce > chiffre_affaires_par_paiement_mapper.py > ...

```python
#!/usr/bin/env python3
import sys

for transaction in sys.stdin:

    # Supprimer le retour à la ligne puis découper la ligne sur les virgules
    fields = transaction.strip().split(",")

    # Ignorer les lignes qui n'ont pas exactement 9 valeurs
    if len(fields) != 9:
        continue

    # Donner un nom à chaque élément de la liste
    transaction_id, date, country, city, product, category, quantity, unit_price, payment_method = fields

    # Ignorer l'en-tête du fichier
    if transaction_id == "transaction_id":
        continue

    # Les champs sont des chaînes : convertir la quantité en entier pour pouvoir calculer
    quantity = int(quantity)

    # Convertir le prix unitaire en centimes entiers (round évite les erreurs de flottants)
    unit_price_cents = round(float(unit_price) * 100)

    # Émettre la paire clé<TAB>valeur : moyen de paiement et montant de la vente en centimes
    print(f"{payment_method}\t{quantity * unit_price_cents}")
```

Le reducer additionne les montants tant que le moyen de paiement ne change pas, grâce au tri des clés effectué par Hadoop avant la réduction.

mapreduce > chiffre_affaires_par_paiement_reducer.py > ...

```python
#!/usr/bin/env python3
import sys

current_payment = None
current_amount = 0

for line in sys.stdin:

    # Séparer la clé et la valeur sur la tabulation
    payment, amount = line.strip().split("\t")

    # Convertir le montant en entier (centimes) pour pouvoir additionner
    amount = int(amount)

    if current_payment == payment:
        # Même moyen de paiement que la ligne précédente : cumuler
        current_amount = current_amount + amount
    else:
        # Nouveau moyen de paiement : afficher le total du précédent
        if current_payment is not None:
            print(f"{current_payment}\t{current_amount}")
        # Démarrer le cumul du nouveau moyen de paiement
        current_payment = payment
        current_amount = amount

# Afficher le total du dernier moyen de paiement
if current_payment is not None:
    print(f"{current_payment}\t{current_amount}")
```

La commande cat | mapper | sort | reducer simule le test local. Elle donne quatre moyens de paiement : BankTransfer, Card, Cash et MobileMoney.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode bash -c "cat /data/transactions.csv
  | python3 /mapreduce/chiffre_affaires_par_paiement_mapper.py | sort | python3 /mapreduce/chiffre_affaires_par_paiement_reducer.
py"
BankTransfer    1400721500
Card    1401016150
Cash    1416870000
MobileMoney     1415720100
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Lancement sur YARN. Le job est soumis avec Hadoop Streaming. Le ResourceManager lance un ApplicationMaster, qui démarre les mappers puis le reducer.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming
-3.4.1.jar -files /mapreduce/chiffre_affaires_par_paiement_mapper.py,/mapreduce/chiffre_affaires_par_paiement_reducer.py -mapper "python3 chiffre_affaires_
par_paiement_mapper.py" -reducer "python3 chiffre_affaires_par_paiement_reducer.py" -input /assignment/transactions/transactions.csv -output /assignment/ou
tput/chiffre_affaires_par_paiement
```

```
2026-11-08 13:32:33 INFO  Job:1754 -  map 100% reduce 100%
2026-11-08 13:32:34 INFO  Job:1765 - Job job_1794125597256_0004 completed successfully
```

Vérification des fichiers. Le dossier de sortie contient _SUCCESS et part-00000.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfs -ls /assignment/output/chiffre_affaires_par_paiement
Found 2 items
-rw-r--r--   2 root supergroup          0 2026-11-08 13:32 /assignment/output/chiffre_affaires_par_paiement/_SUCCESS
-rw-r--r--   2 root supergroup         79 2026-11-08 13:32 /assignment/output/chiffre_affaires_par_paiement/part-00000
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Résultat. Les totaux par moyen de paiement, en centimes, sont lus dans HDFS.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfs -cat /assignment/output/chiffre_affaires_par_paiement
/part-00000
BankTransfer    1400721500
Card    1401016150
Cash    1416870000
MobileMoney     1415720100
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Validation indépendante. Le script validate.py paiement recalcule les totaux depuis transactions.csv, sans MapReduce. Ses résultats sont identiques à ceux du job comme le montre la capture suivante :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode python3 /mapreduce/validate.py paiement
BankTransfer    1400721500
Card    1401016150
Cash    1416870000
MobileMoney     1415720100
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

4. job 4 : chiffre d'affaires par mois

Pour calculer le chiffre d'affaires par mois, les étapes suivantes sont réalisées.Le mapper ignore l'en-tête et les lignes mal formées, extrait le mois (AAAA-MM) des 7 premiers caractères de la date, convertit le prix en centimes entiers et émet mois<TAB>montant en centimes.

```python
import sys
for transaction in sys.stdin:

    # Supprimer le retour à la ligne puis découper la ligne sur les virgules
    fields = transaction.strip().split(",")

    # Ignorer les lignes qui n'ont pas exactement 9 valeurs
    if len(fields) != 9:
        continue

    # Donner un nom à chaque élément de la liste
    transaction_id, date, country, city, product, category, quantity, unit_price, payment_method = fields

    # Ignorer l'en-tête du fichier
    if transaction_id == "transaction_id":
        continue


    #Extraire le mois (AAAA-MM) des 7 premiers caractères de la date
    month = date[:7]

    # Les champs sont des chaînes : convertir la quantité en entier pour pouvoir calculer
    quantity = int(quantity)

    # Convertir le prix unitaire en centimes entiers (round évite les erreurs de flottants)
    unit_price_cents = round(float(unit_price) * 100)

    # Émettre la paire clé<TAB>valeur : mois et montant de la vente en centimes
    print(f"{month}\t{quantity * unit_price_cents}")
```

Le reducer additionne les montants tant que le mois ne change pas, grâce au tri des clés effectué par Hadoop avant la réduction.

```python
#!/usr/bin/env python3
import sys

current_month = None
current_amount = 0

for line in sys.stdin:

    # Séparer la clé et la valeur sur la tabulation
    month, amount = line.strip().split("\t")

    # Convertir le montant en entier (centimes) pour pouvoir additionner
    amount = int(amount)

    if current_month == month:
        # Même mois que la ligne précédente : cumuler
        current_amount = current_amount + amount
    else:
        # Nouveau mois : afficher le total du mois précédent
        if current_month is not None:
            print(f"{current_month}\t{current_amount}")
        # Démarrer le cumul du nouveau mois
        current_month = month
        current_amount = amount

# Afficher le total du dernier mois
if current_month is not None:
    print(f"{current_month}\t{current_amount}")
```

La commande cat | mapper | sort | reducer simule le test local. Elle donne douze mois, de 2025-01 à 2025-12, triés chronologiquement.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode bash -c "cat /data/transactions.csv | python3 /mapreduc
e/chiffre_affaires_par_mois_mapper.py | sort | python3 /mapreduce/chiffre_affaires_par_mois_reducer.py"
```

```
2025-01 472167000
2025-02 440945550
2025-03 489890550
2025-04 459014950
2025-05 469357850
2025-06 473010400
2025-07 473184350
2025-08 474458400
2025-09 462934650
2025-10 468376200
```

Lancement sur YARN. Le job est soumis avec Hadoop Streaming. Le ResourceManager lance un ApplicationMaster, qui démarre les mappers puis le reducer.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-st
reaming-3.4.1.jar -files /mapreduce/chiffre_affaires_par_mois_mapper.py,/mapreduce/chiffre_affaires_par_mois_reducer.py -mapper "python3 chiffre_aff
aires_par_mois_mapper.py" -reducer "python3 chiffre_affaires_par_mois_reducer.py" -input /assignment/transactions/transactions.csv -output /assignme
nt/output/chiffre_affaires_par_mois
```

```
2026-11-08 14:05:09 INFO  Job:1765 - Job job_1794125597256_0005 completed successfully
```

Vérification des fichiers. Le dossier de sortie contient _SUCCESS et part-00000.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfs -ls /assignment/output/chiffre_affaires_par_mo
is
Found 2 items
-rw-r--r--   2 root supergroup          0 2026-11-08 14:05 /assignment/output/chiffre_affaires_par_mois/_SUCCESS
-rw-r--r--   2 root supergroup        216 2026-11-08 14:05 /assignment/output/chiffre_affaires_par_mois/part-00000
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Résultat. Les totaux par mois, en centimes, sont lus dans HDFS.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfs -cat /assignment/output/chiffre_affaires_par_m
ois/part-00000
2025-01 472167000
2025-02 440945550
2025-03 489890550
2025-04 459014950
2025-05 469357850
2025-06 473010400
2025-07 473184350
2025-08 474458400
2025-09 462934650
2025-10 468376200
2025-11 464793050
2025-12 486194800
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Validation indépendante. Le script validate.py mois recalcule les totaux depuis transactions.csv, sans MapReduce. Ses résultats sont identiques à ceux du job.

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode python3 /mapreduce/validate.py mois
2025-01 472167000
2025-02 440945550
2025-03 489890550
2025-04 459014950
2025-05 469357850
2025-06 473010400
2025-07 473184350
2025-08 474458400
2025-09 462934650
2025-10 468376200
2025-11 464793050
2025-12 486194800
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Partie D — Réplication et panne

D.1 État de référence avec 3 DataNodes

On vérifie l'état de référence avec la commande hdfs dfsadmin -report :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfsadmin -report | Select-String "Live datanodes|N
ame:|Hostname:"

Live datanodes (3):
Name: 172.19.0.3:9866 (hadoop-datanode1.assignment1_default)
Hostname: datanode1
Name: 172.19.0.4:9866 (hadoop-datanode3.assignment1_default)
Hostname: datanode3
Name: 172.19.0.6:9866 (hadoop-datanode2.assignment1_default)
Hostname: datanode2
```

Avant la panne, le NameNode voit trois DataNodes actifs : datanode1 (172.19.0.3), datanode3 (172.19.0.4) et datanode2 (172.19.0.6).

On vérifie ensuite l'état du fichier avec hdfs fsck :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs fsck /assignment/transactions/transactions.csv -files -blocks -locations
Connecting to namenode via http://namenode:9870/fsck?ugi=root&files=1&blocks=1&locations=1&path=%2Fassignment%2Ftransactions%2Ftransactions.csv
FSCK started by root (auth:SIMPLE) from /172.19.0.2 for path /assignment/transactions/transactions.csv at Sun Nov 08 15:10:36 UTC 2026
```

```
/assignment/transactions/transactions.csv 7109550 bytes, replicated: replication=2, 7 block(s):  OK
0. BP-165148381-172.19.0.2-1793924235033:blk_1073741832_1008 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.3:9866,DS-7a1c624b-ddcd-49c3-9933-536a6b58dd0c,DISK], DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK]]
1. BP-165148381-172.19.0.2-1793924235033:blk_1073741833_1009 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
2. BP-165148381-172.19.0.2-1793924235033:blk_1073741834_1010 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.3:9866,DS-7a1c624b-ddcd-49c3-9933-536a6b58dd0c,DISK], DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK]]
3. BP-165148381-172.19.0.2-1793924235033:blk_1073741835_1011 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
4. BP-165148381-172.19.0.2-1793924235033:blk_1073741836_1012 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.3:9866,DS-7a1c624b-ddcd-49c3-9933-536a6b58dd0c,DISK], DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK]]
5. BP-165148381-172.19.0.2-1793924235033:blk_1073741837_1013 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
6. BP-165148381-172.19.0.2-1793924235033:blk_1073741838_1014 len=818094 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.3:9866,DS-7a1c624b-ddcd-49c3-9933-536a6b58dd0c,DISK], DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK]]


Status: HEALTHY
 Number of data-nodes:  3
 Number of racks:               1
 Total dirs:                    0
 Total symlinks:                0
```

Le fichier compte 7 blocs, chacun avec 2 répliques vivantes (Live_repl=2) par exemple pour le bloc 0 il se trouve dans le datanode1 et datanode2. Son état est HEALTHY. Les blocs sont répartis sur les trois DataNodes.

D.2 Arrêt volontaire de datanode1

On arrête datanode1 pour simuler une panne :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker stop hadoop-datanode1
hadoop-datanode1
```

Le NameNode ne le déclare pas mort tout de suite : il attend environ 40 . Après ce délai, un nouveau rapport montre deux DataNodes vivants et datanode1 déclaré mort :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfsadmin -report | Select-String "Live datanodes|Dead datanodes|Name:|Hostname:"

Live datanodes (2):
Name: 172.19.0.4:9866 (hadoop-datanode3.assignment1_default)
Hostname: datanode3
Name: 172.19.0.6:9866 (hadoop-datanode2.assignment1_default)
Hostname: datanode2
Dead datanodes (1):
Name: 172.19.0.3:9866 (172.19.0.3)
Hostname: datanode1

PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Avec fsck, l'adresse 172.19.0.3 n'apparaît plus : les 7 blocs sont sur datanode2 et datanode3, et chacun a retrouvé 2 répliques :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs fsck /assignment/transactions/transactions.csv -fi
les -blocks -locations
Connecting to namenode via http://namenode:9870/fsck?ugi=root&files=1&blocks=1&locations=1&path=%2Fassignment%2Ftransactions%2Ftransactions.csv
FSCK started by root (auth:SIMPLE) from /172.19.0.2 for path /assignment/transactions/transactions.csv at Sun Nov 08 15:39:02 UTC 2026

/assignment/transactions/transactions.csv 7109550 bytes, replicated: replication=2, 7 block(s):  OK
0. BP-165148381-172.19.0.2-1793924235033:blk_1073741832_1008 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05
-85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
1. BP-165148381-172.19.0.2-1793924235033:blk_1073741833_1009 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05
-85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
2. BP-165148381-172.19.0.2-1793924235033:blk_1073741834_1010 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05
-85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
3. BP-165148381-172.19.0.2-1793924235033:blk_1073741835_1011 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05
-85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
4. BP-165148381-172.19.0.2-1793924235033:blk_1073741836_1012 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05
-85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
5. BP-165148381-172.19.0.2-1793924235033:blk_1073741837_1013 len=1048576 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05
-85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
6. BP-165148381-172.19.0.2-1793924235033:blk_1073741838_1014 len=818094 Live_repl=2  [DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-
85b6-e6b2c5ad5676,DISK], DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]


Status: HEALTHY
 Number of data-nodes:  2
 Number of racks:               1
 Total dirs:                    0
 Total symlinks:                0
```

Quand datanode1 est tombé, les 4 blocs qu'il portait n'avaient plus qu'une réplique (sous-répliqués). HDFS les a recopiés sur datanode2. À notre observation, cette re-réplication était déjà terminée, donc aucun bloc à Live_repl=1 n'a été capturé. Aucun bloc n'est manquant.

Malgré la panne, le fichier reste accessible, car chaque bloc avait une copie sur un autre DataNode :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfs -cat /assignment/transactions/transactions.cs
 | Select-Object -First 3
transaction_id,date,country,city,product,category,quantity,unit_price,payment_method
T0000001,2025-11-28,Senegal,Thies,Backpack,Accessories,5,60.00,Card
T0000002,2025-08-04,France,Marseille,Headphones,Accessories,5,90.00,MobileMoney
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Depuis le ResourceManager, on vérifie que les NodeManagers ne sont pas affectés par la panne d'un DataNode : les trois sont RUNNING :

D.3 Restauration de datanode1

On redémarre datanode1 avec docker start et on vérifie le rapport :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker start hadoop-datanode1
hadoop-datanode1
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfsadmin -report | Select-String "Live datanodes|D
ead datanodes|Name:|Hostname:"

Live datanodes (3):
Name: 172.19.0.3:9866 (hadoop-datanode1.assignment1_default)
Hostname: datanode1
Name: 172.19.0.4:9866 (hadoop-datanode3.assignment1_default)
Hostname: datanode3
Name: 172.19.0.6:9866 (hadoop-datanode2.assignment1_default)
Hostname: datanode2
```

Avec fsck, on constate que les blocs restent uniquement sur datanode2 et datanode3

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs fsck /assignment/transactions/transactions.csv -f
les -blocks -locations | Select-String "Over-replicated|Under-replicated|Number of data-nodes|Status|Live_repl"
Connecting to namenode via http://namenode:9870/fsck?ugi=root&files=1&blocks=1&locations=1&path=%2Fassignment%2Ftransactions%2Ftransactions.csv

0. BP-165148381-172.19.0.2-1793924235033:blk_1073741832_1008 len=1048576 Live_repl=2
[DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK],
DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
1. BP-165148381-172.19.0.2-1793924235033:blk_1073741833_1009 len=1048576 Live_repl=2
[DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK],
DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
2. BP-165148381-172.19.0.2-1793924235033:blk_1073741834_1010 len=1048576 Live_repl=2
[DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK],
DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
3. BP-165148381-172.19.0.2-1793924235033:blk_1073741835_1011 len=1048576 Live_repl=2
[DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK],
DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
4. BP-165148381-172.19.0.2-1793924235033:blk_1073741836_1012 len=1048576 Live_repl=2
[DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK],
DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
5. BP-165148381-172.19.0.2-1793924235033:blk_1073741837_1013 len=1048576 Live_repl=2
[DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK],
DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
6. BP-165148381-172.19.0.2-1793924235033:blk_1073741838_1014 len=818094 Live_repl=2
[DatanodeInfoWithStorage[172.19.0.4:9866,DS-bd787edd-aa16-4d05-85b6-e6b2c5ad5676,DISK],
DatanodeInfoWithStorage[172.19.0.6:9866,DS-43101419-2bd0-43bf-a86d-d52b28cacf2c,DISK]]
Status: HEALTHY
 Number of data-nodes:  3
```

Explication : chaque bloc a déjà ses 2 répliques demandées, donc HDFS n'a aucune raison d'en créer une troisième sur le nœud qui revient.

D.4 Panne d'un NodeManager

On arrête nodemanager1 :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker stop hadoop-nodemanager1
hadoop-nodemanager1
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Avec docker ps, on voit que seuls nodemanager2 et nodemanager3 restent actifs :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker ps --filter name=hadoop-nodemanager
CONTAINER ID   IMAGE                  COMMAND                  CREATED      STATUS       PORTS     NAMES
8a6266f50f46   hadoop-python:3.4.1    "/usr/local/bin/dumb…"   2 days ago   Up 8 hours             hadoop-nodemanager3
8062d1dea7df   hadoop-python:3.4.1    "/usr/local/bin/dumb…"   2 days ago   Up 8 hours             hadoop-nodemanager2
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Juste après l'arrêt, le ResourceManager affiche encore nodemanager1 en RUNNING. En effet , nous n'avons pas réduit le délai de détection de YARN, contrairement à celui de HDFS, donc le nœud n'est déclaré perdu qu'après un long délai (10 minutes et 30s par défaut) .

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-resourcemanager yarn node -list
2026-11-08 15:44:59 INFO  DefaultNoHARMFailoverProxyProvider:64 - Connecting to ResourceManager at resourcemanager/172.19.0.5:8032
Total Nodes:3
         Node-Id             Node-State Node-Http-Address   Number-of-Running-Containers
nodemanager1:36895               RUNNING nodemanager1:8042                             0
nodemanager2:40615               RUNNING nodemanager2:8042                             0
nodemanager3:46109               RUNNING nodemanager3:8042                             0
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Pour savoir si cette panne affecte le calcul, on relance le job 1 :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfs -rm -r -f /assignment/output/chiffre_affaires_
par_pays
Deleted /assignment/output/chiffre_affaires_par_pays
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-st
reaming-3.4.1.jar -files /mapreduce/chiffre_affaires_par_pays_mapper.py,/mapreduce/chiffre_affaires_par_pays_reducer.py -mapper "python3 chiffre_aff
aires_par_pays_mapper.py" -reducer "python3 chiffre_affaires_par_pays_reducer.py" -input /assignment/transactions/transactions.csv -output /assignme
nt/output/chiffre_affaires_par_pays
```

Le job se termine avec succès. En effet les données sont sur les DataNodes, qui n'ont pas été touchés, et le calcul a utilisé les NodeManagers restants, qui avaient assez de mémoire et de vcores pour les containers (ApplicationMaster, mappers, reducer). On récupère la sortie du job :

```
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1> docker exec hadoop-namenode hdfs dfs -cat /assignment/output/chiffre_affaires_par_p
ays/part-00000
Canada  1123162050
France  1120959850
Germany 1138438050
Morocco 1127850500
Senegal 1123917300
PS C:\Users\pc\Desktop\Ing1\S2\Big Data\Assignment1\Assignment1>
```

Les 5 lignes sont identiques à celles du premier lancement.

D.5 Distinction DataNode / NodeManager

Un DataNode stocke des blocs : sa panne réduit le nombre de répliques, que HDFS reconstitue ailleurs, sans effet sur YARN. Un NodeManager exécute des tâches : sa panne réduit la capacité de calcul, sans effet sur les données HDFS.

D.6 Limites d'un cluster Docker sur un seul PC

Les huit conteneurs partagent le même processeur, la même mémoire, le même disque et la même alimentation. Une panne du PC les emporte tous : la tolérance aux pannes est donc simulée, pas réelle.

7. Questions de compréhension

1) Pourquoi le NameNode ne stocke-t-il pas les blocs ?

Le NameNode gère uniquement les métadonnées : l'arborescence, les blocs de chaque fichier et leur emplacement. Il les garde en mémoire pour répondre vite. S'il stockait aussi les données, il deviendrait un goulot d'étranglement et sa mémoire serait saturée. En confiant les blocs aux DataNodes, le stockage et les lectures se répartissent sur plusieurs machines. Dans notre cluster, le fsck montre que les blocs sont sur 172.19.0.3, .4 et .6 (les DataNodes), jamais sur le NameNode.

2) Pourquoi exécuter hdfs dfs -put depuis le conteneur NameNode ne signifie-t-il pas que les blocs y sont stockés ?

Le conteneur NameNode sert seulement de poste de travail : il contient le client HDFS et voit le fichier via le montage /data. Le client demande au NameNode où écrire chaque bloc, puis envoie les données directement aux DataNodes désignés. Preuve dans notre projet : après le put, fsck -locations indique que les 7 blocs sont sur des DataNodes (172.19.0.3, .4, .6), pas sur le NameNode.

3) Que signifie « réplication = 2 » avec trois DataNodes ?

Chaque bloc est conservé en 2 copies, placées sur 2 DataNodes différents parmi les 3. Les trois nœuds ne portent donc pas tous chaque bloc : chaque bloc manque sur un des trois. Notre fsck le montre, avec Live_repl=2 partout et des paires différentes selon les blocs. Cela permet de perdre un DataNode sans perdre de données, et c'est ce que montre la Partie D : après l'arrêt de datanode1, le fichier reste lisible et les blocs sont recopiés pour retrouver 2 répliques. Le coût est un stockage double (environ 14 Mio pour 6,78 Mio de données).

4) Quelle différence entre un DataNode et un NodeManager ?

Un DataNode appartient à HDFS : il stocke les blocs et envoie des heartbeats au NameNode. Un NodeManager appartient à YARN : il exécute les containers (ApplicationMaster, mappers, reducers) et rapporte ses ressources au ResourceManager. Ce sont deux couches indépendantes, dans des conteneurs Docker séparés. La Partie D le confirme : l'arrêt de datanode1 n'a pas modifié yarn node -list, et l'arrêt de nodemanager1 a laissé Live datanodes (3).

5) Pourquoi plusieurs conteneurs sur un PC ne sont-ils pas équivalents à plusieurs serveurs physiques ?

Les huit conteneurs partagent le même processeur, la même mémoire, le même noyau et le même disque physique : les 2,95 To vus par chaque DataNode sont un seul disque. Une panne du PC, du disque ou de l'alimentation emporte tous les nœuds à la fois, donc la tolérance aux pannes est simulée et non réelle. Il n'y a pas non plus de vrai réseau entre les nœuds (pas de latence ni de panne réseau indépendante), et les performances ne s'additionnent pas puisque tous se disputent les mêmes ressources. Dans la Partie D, nous avons arrêté un conteneur, ce qui simule la panne d'un nœud, mais pas celle d'une machine entière.